Implement the backend functionality for favourites and targeted in-app notifica…

I'll read the pasted requirements and check the current backend, migrations, tests, and design before changing anything. I'll keep this backend only and follow the repository's test first commit workflow.

Baseline is clean: 76 backend tests and the backend build pass. The test database has migrations 001–004, and neither new table exists yet. I'll add the schema test first, then create migration 005.

Edited files

Migration 005 now passes in PostgreSQL and remains idempotent under the migration runner. Next I'm testing the owner scoped favourites API, including zero-stock saves and soft-deleted saved vehicles.

Edited files

Favourites now support save, list, removal, and explicit restock preference updates. The passing tests cover owner isolation, duplicate saves, zero stock, and preserving an existing preference. I'm adding the notification inbox and mark-read API next.

Edited files

The inbox and read-status API passes its owner isolation and pagination tests. I'm now wiring alert creation into the stock transactions. The first event test checks the exact low-stock boundary and active admin targeting.

Edited files

Direct purchase and checkout now create admin alerts inside their existing transactions, and the focused purchase regressions still pass. I'm testing restock targeting next: only active accounts with a current favourite and explicit opt-in should receive the zero-to-positive alert.

Edited files

The restock alert logic builds, but the combined integration run exposed a test isolation issue: another suite's active admin also receives the alerts, as the product rules require, and its teardown did not account for the new notification foreign key. I'm updating those fixtures and assertions, then I'll rerun both suites.

Edited files

The restock targeting test now passes, and the affected suites pass together. I'm testing transaction failure paths next with a PostgreSQL trigger that deliberately rejects an alert insert, so I can verify stock, history, and cart state all roll back.

Edited files

The rollback test passed for direct purchase, cart checkout, and restock. I also verified that removing a favourite stops future restock alerts while keeping earlier alerts, and added database constraint checks. I'm running the full backend suite now to catch interactions between test files.

The full suite passes at 85 tests. A fixture audit found rows left by the earlier failed cross-suite run: six alert test vehicles and two inventory test users. I've identified their exact prefixes and will remove only those rows from the dedicated _test database.

Edited files

You've hit your usage limit. Contact your workspace owner for more access.

Edited 14 files+513-5View changesUndoView changes

backend/test/integration/migrate.integration.test.tsbackend/test/integration/migrate.integration.test.ts+43-0 backend/migrations/005_create_favourites_and_notifications.sqlbackend/migrations/005_create_favourites_and_notifications.sql +23-0 backend/test/migrate.test.tsbackend/test/migrate.test.ts+2-0 Show 11 more files

continue

6:21 PM

Worked for 9m 38s

Implemented the complete backend favourites and targeted notification functionality. TDD commits Cycle RED GREEN Schema 4055fda 3689492 Favourite list/save 60938ca 4989bbd Preference/remove ccdf8e3 1453311 Notification inbox/read 35def0d 1a425b5 Low-stock alerts 42ad9d4 eb5644d Out-of-stock/checkout alerts c18d20f 7a70cb9 Back-in-stock alerts 2f0ede4 76af98a

Additional commits:

- bbf6cb2 — rollback, targeting, and PostgreSQL constraint verification
- f807dae — API documentation and exact prompt history
- 3a5612d — deterministic PostgreSQL test isolation No REFACTOR commit was necessary. Implemented
- Owner-scoped favourite add, list, delete, and alert preference APIs
- Zero-stock favourites and retained soft-deleted favourites
- Paginated private notification inbox with unread count
- Owner-only, idempotent mark-read behavior
- Active-admin low_stock and out_of_stock alerts
- Opted-in back_in_stock alerts for active users
- Stable event keys and database duplicate protection
- Notification creation inside purchase, checkout, and restock transactions
- Full rollback of stock, history, cart state, and notifications on failure
- Migration 005 with constraints and required indexes The contracts are documented in [README.md](C:/PROJECTS/Dealership Inventory/README.md) and the schema in [database-design.md](C:/PROJECTS/Dealership Inventory/docs/database-design.md). The exact prompt was appended to [PROMPTS.md](C:/PROJECTS/Dealership Inventory/PROMPTS.md). Verification
- Baseline: 76 tests passed
- Final backend suite: 85/85 passed
- Backend TypeScript build passed
- Root backend/frontend build passed
- Migration 005 applied successfully
- Second migration run reported no pending migrations
- git diff --check passed
- Test fixture audit: zero leftover users and vehicles
- No frontend files changed
- Final status: main...origin/main [ahead 17], clean working tree

- Nothing was pushed to GitHub Skipped: frontend implementation and vehicle dataset work, as requested. Risk: integration test files now run serially to prevent shared PostgreSQL admin fixtures from receiving each other's alerts; this makes the suite slower but deterministic.

  Edited 2 files+5-2View changesUndoView changes

backend/package.jsonbackend/package.json+1-1
backend/test/integration/vehicles.integration.test.tsbackend/test/integration/vehicles.integration.test.ts+4-1

6:31 PM

push it on github

6:34 PM

Worked for 1m 25s

Pushed successfully to GitHub. main and origin/main now match at commit 3a5612d. The working tree is clean, and no force push or history rewrite was used.

6:36 PM

I want to finish the Car Dealership Inventory System so I can run the complete …

I'm using the Ponytail coding workflow again because this is repository implementation work: I'll read the exact pasted request and current repo state first, then make the smallest complete, test-backed changes it requires.

The repository contains Gemini's dataset/image commits and is clean, with 12 local commits ahead of GitHub. I'm locating the UI pack now; if it is present, I'll inspect every required design/API file before touching the frontend.

The UI pack is available at reference-docs/Creaseline_Frontend_UI_Pack.zip, so there is no blocker. I'm extracting it into a temporary ignored workspace, then reading the specifications and inspecting the reference screens before auditing Gemini's code and database.

The pack confirms the real scope and API gaps: no category-list, purchase-history, receipt-history, or admin-summary endpoint, so the UI must stay honest around those areas. I'm visually inspecting the seven Stitch screens now, then I'll audit Gemini's dataset, seeder, images, static route, and live database before frontend work begins.

Context automatically compacted

I've finished mapping the UI pack and am now resuming the dataset/image audit in smaller, verifiable slices. I'll keep the provider untouched while checking determinism, secrets, database reconciliation, and the existing test baseline.

The static audit has already confirmed the 10,000-row source produces 35 deterministic, unique catalogue entries and the checked-in seed matches byte-for-byte at the data level. I also found one area that needs closer proof: the curator claims category balance, but the actual sample is uneven, so I'm testing whether that is a real requirement defect before changing it.

The live development database matches the expected safe checkpoint: 35 active vehicles, 27 opening receipts, 16 image rows, and zero stock-reconciliation mismatches. All 16 referenced WebP files exist, are non-empty, unique by hash, and have valid WebP signatures; no duplicate or malformed image rows were found.

The baseline is clean: all 101 backend tests pass, the full workspace builds, and the migration runner reports no pending migrations. I did identify one genuine safety defect: catalogue seeding begins its transaction only after it mutates canonical categories and potentially the system actor. I'm adding a focused RED test for true all-or-nothing setup before making the smallest transaction-boundary fix.

Edited files

The seeder defect is now fixed with visible TDD history: RED 6986935, GREEN a6032b2. The focused unit test and real PostgreSQL seeder integration tests pass, including the rerun/no-stock-reset behavior. I'm finishing the image delivery and provider-contract checks next, without calling the provider.

Searched the web for pricing free evaluation personal use terms commercial carimage.dev | carimage.dev ...

The audit exit condition is satisfied: the catalogue is deterministic and reconciled, cached image delivery works from the compiled backend (200 image/webp), traversal is rejected, and current provider terms confirm free images are evaluation/personal-only while commercial use requires a paid plan. I'm now starting the frontend with the smallest justified additions: routing plus Vitest/DOM testing tools.

The package manager is taking unusually long without output, likely on registry access. I'm giving it one short final window; if it remains stalled I'll stop it and use an approved network retry rather than leave the workspace in an uncertain install state.

Routing installed successfully. The test-tool install is still quiet but active; I'm monitoring it closely and will only proceed once the lockfile and dependency tree are consistent. The audit also surfaced two pre-existing high-severity npm advisories, which I'll inspect separately instead of applying an indiscriminate audit fix.

Edited files

The first frontend slice is complete with TDD: RED 5f0c2a2, GREEN d05d8c4. The shared client now covers bearer authorization, configurable base URLs, network/malformed-response handling, session expiry, and preserved 409 details. Next I'm driving authentication and protected catalogue navigation from a browser-level test.

Edited files

Authentication and protected routing are now GREEN in 69224c5; all four frontend tests and the production build pass. I'm moving to the required backend read APIs next so purchase history, category filters, stock history, and admin summary are real—not inferred or mocked in the UI.

Edited files

The vehicle card slice is GREEN in 5446800: real INR prices, truthful low/zero-stock states, neutral local fallback imagery, disabled purchase actions at zero stock, and the signature fold only after a confirmed server add. I'm now locking down checkout retry/idempotency before wiring the cart screen.

Edited files

Checkout retry behavior is now GREEN in a5361f3, and the user workflow pages are being wired directly to the real services. The catalogue, detail, cart, favourites, notifications, and purchase ledger are in place; I'm finishing the admin inventory console and then applying the Creaseline responsive visual system.

Edited files

The complete UI workflow now builds and all frontend tests pass: authenticated catalogue/detail, cart checkout, favourites, notifications, purchase history, and the admin inventory console are all connected to real APIs with responsive Creaseline styling. I'm moving through run tooling, documentation, prompt preservation, and full end-to-end verification now.

Edited files

You've hit your usage limit. Contact your workspace owner for more access.

Edited 44 files+1,694-83View changesUndoView changes

backend/test/seeder.test.tsbackend/test/seeder.test.ts+58-0 backend/src/seeder.tsbackend/src/seeder.ts+22-22 frontend/package.jsonfrontend/package.json+1-0 Show 41 more files

continue

9:10 PM

Worked for 31m 59s

Completed the Car Dealership Inventory System.

- Built the full responsive React frontend: authentication, catalogue, vehicle details, cart/checkout, favourites, notifications, purchase history, and admin inventory workflows.
- Added missing backend read APIs for categories, purchase history, stock receipts, and inventory summaries.
- Preserved transactional/idempotent checkout, stock integrity, server-side permissions, and secret handling.
- Audited the dataset: 10,000 rows processed, 20 rejected, 35 deterministic vehicle lines seeded.
- Verified 35 active vehicles, 27 opening receipts, 16 valid cached images, and zero stock reconciliation or reference errors.
- Fixed checkout conflicts so refreshed stock no longer erases the shopper's guidance.
- Updated setup and operating instructions in [README.md](C:/PROJECTS/Dealership Inventory/README.md). Verification:
- Backend: 106 tests passed across 16 files.
- Frontend: 12 tests passed across 5 files.
- Production frontend build passed.
- Dependency audit: 0 vulnerabilities.
- Working tree is clean.
- main is 28 commits ahead of origin/main.
- Final TDD pair: 19af3f5 RED, b9911cf GREEN. Not pushed to GitHub. Manual visual browser QA was unavailable because local browser control is disabled. No live image-provider calls were made. CarImage.dev requires an appropriate paid plan for commercial use and warns that generated renders may not be OEM-accurate; the original dataset's redistribution license should also be confirmed before public release. Pricing · Terms

  Edited 6 files+147-94View changesUndoView changes

scripts/dev.mjsscripts/dev.mjs+61-90 package.jsonpackage.json+4-1 frontend/src/features/auth/auth-flow.test.tsxfrontend/src/features/auth/auth-flow.test.tsx+13-0 Show 3 more files

9:42 PM